



Algorithmen und Datenstrukturen

Leon Johann Brettin



Algorithmen und Datenstrukturen

5. Verkettete Listen



Datenstrukturen – Beispiele

- Grundlegende Datenstrukturen:
 - Kollektion, Stack, Warteschlange, **Liste**, ...
 - Zahlreiche mögliche Anwendungen
 - Oft Grundlage für speziellere Datenstrukturen
- Indexstrukturen:
 - Bäume und Hash-Tabellen
 - Effizienter Zugriff auf große Datenmengen
- Komplexe Netzwerke:
 - Grundlage: Graphen
 - Verschiedene interne Darstellungsformen
- Anwendungsspezifische Datenstrukturen
 - Elektrische Schaltungen, Genom, Multimedia, ...

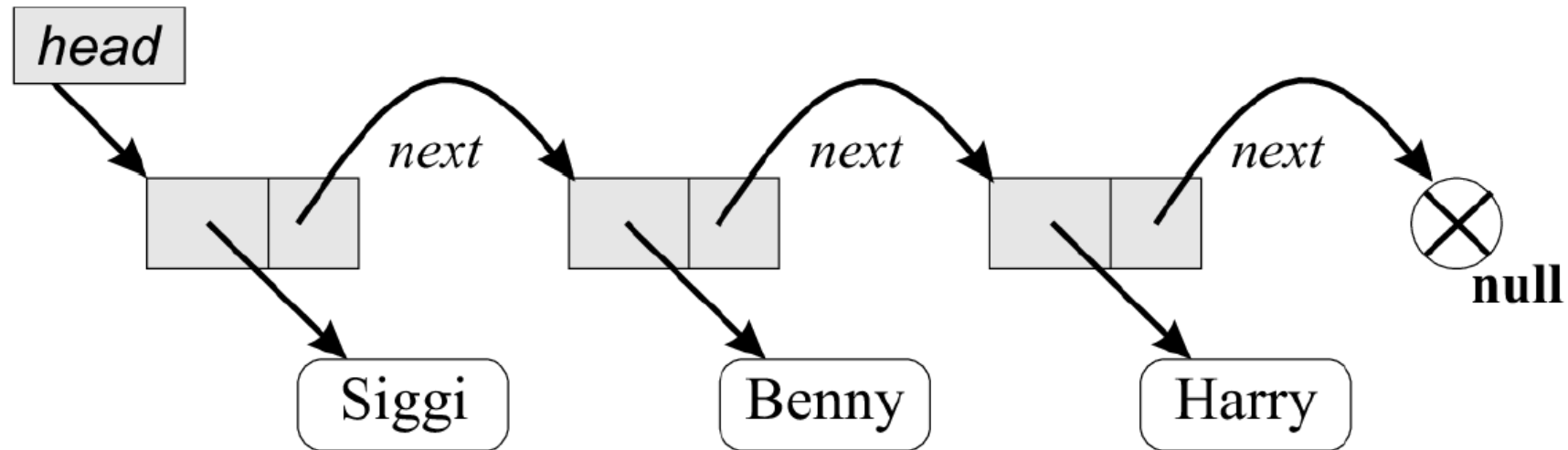


Verkettete Liste

- Bisherige Datenstrukturen: **statisch**
 - können zur Laufzeit nicht wachsen bzw. schrumpfen, abgesehen vom Umkopieren (wachsen ok, schrumpfen?)
 - dadurch keine Anpassung an tatsächlichen Speicherbedarf
- Ausweg: **dynamische** Datenstrukturen
- Beispiel: **verkettete Liste**
 - Menge von Knoten, die untereinander „verzeigert“ sind
 - jeder Knoten besitzt Verweis auf Nachfolgerknoten sowie das zu speichernde Element
 - Listenkopf: spezieller Knoten head
 - Listenende: null-Zeiger



Verkettete Liste





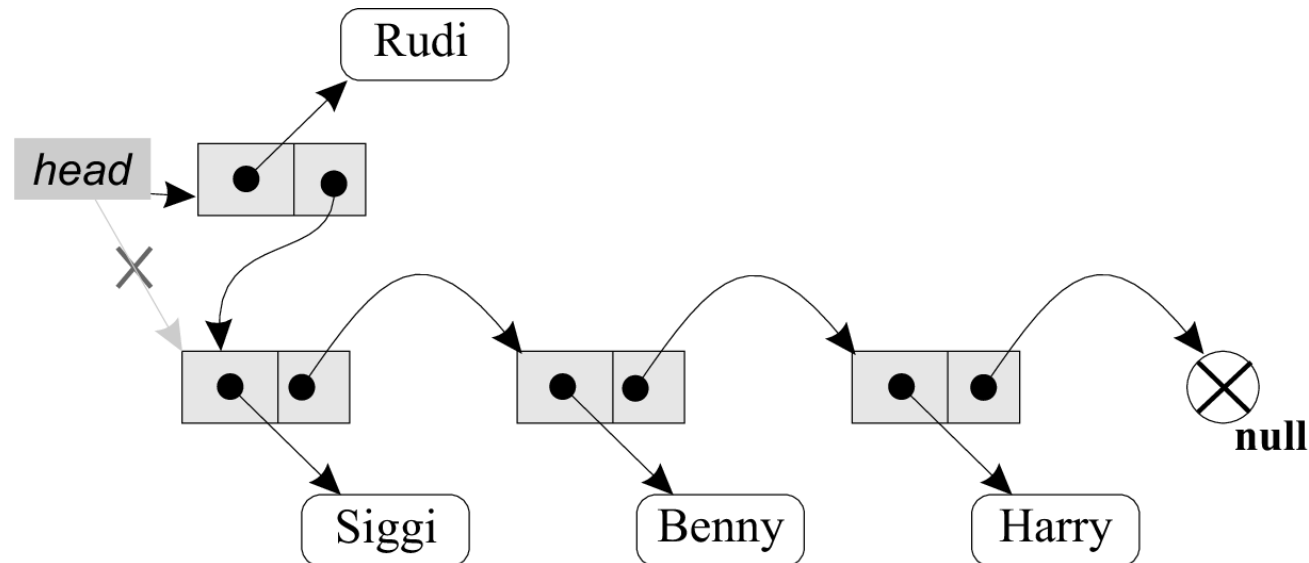
Verkettete Liste: Schnittstelle

- Betrachten wir zunächst die für die Datenstrukturen **Stack** und **Queue** (*schauen wir uns später genauer an*) nötigen Operationen.
 - `void addFirst(Object obj)` fügt das Objekt `obj` als erstes Element in die Liste ein
 - `Object getFirst()` liefert das erste Element der Liste
 - `Object removeFirst()` entfernt das erste Element der Liste und gibt es gleichzeitig als Ergebnis zurück
 - `void addLast(Object obj)` hängt das Objekt `obj` als letztes Element an die Liste an
 - `Object getLast()` liefert das letzte Element der Liste
 - `Object removeLast()` entfernt das letzte Element der Liste und gibt es gleichzeitig als Ergebnis zurück



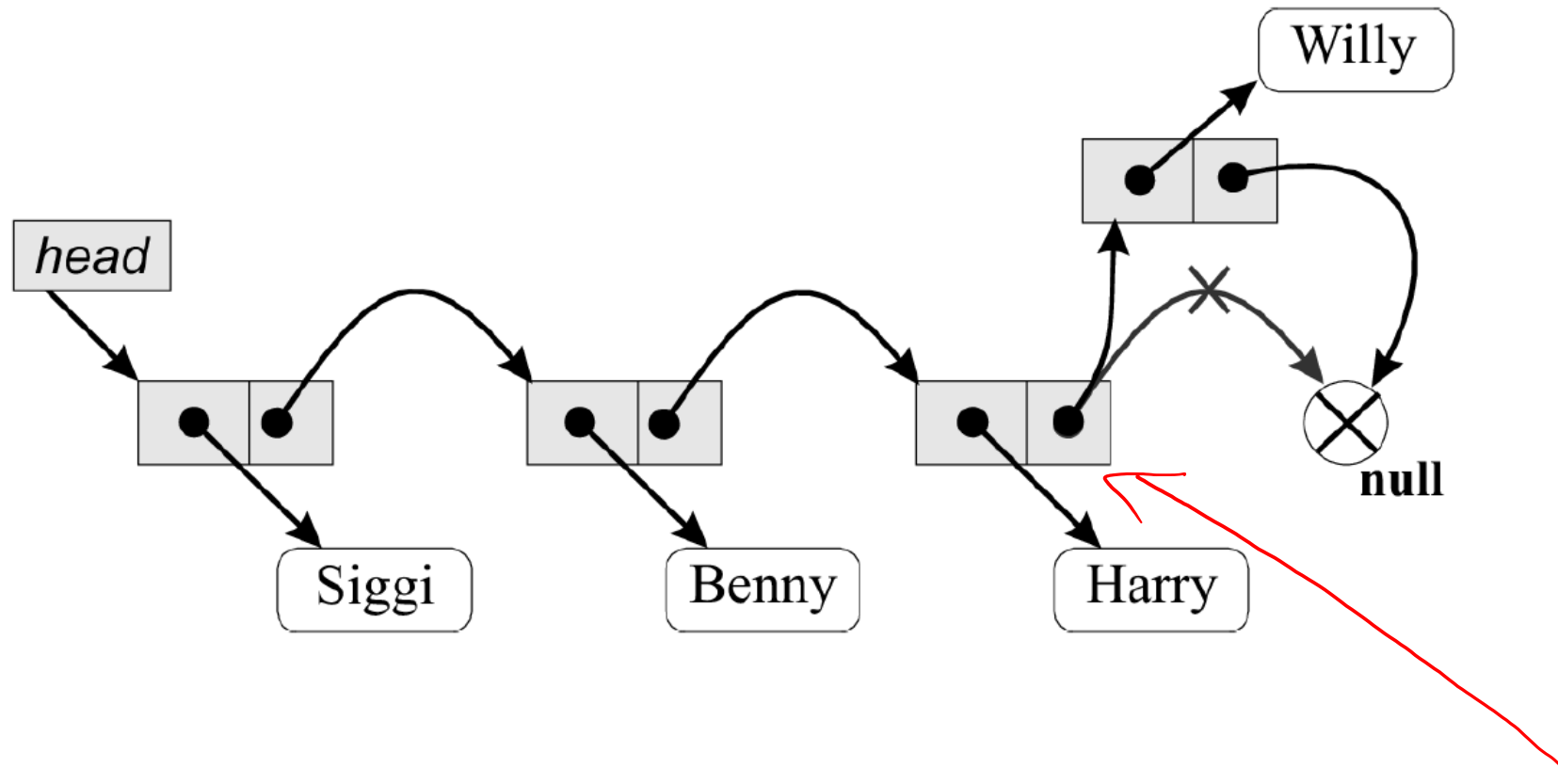
Verkettete Liste: Einfügen

1. Einfügeposition suchen
2. neuen Knoten anlegen
3. Zeiger vom neuen Knoten auf Knoten an Einfügeposition
4. Zeiger vom Vorgänger auf neuen Knoten



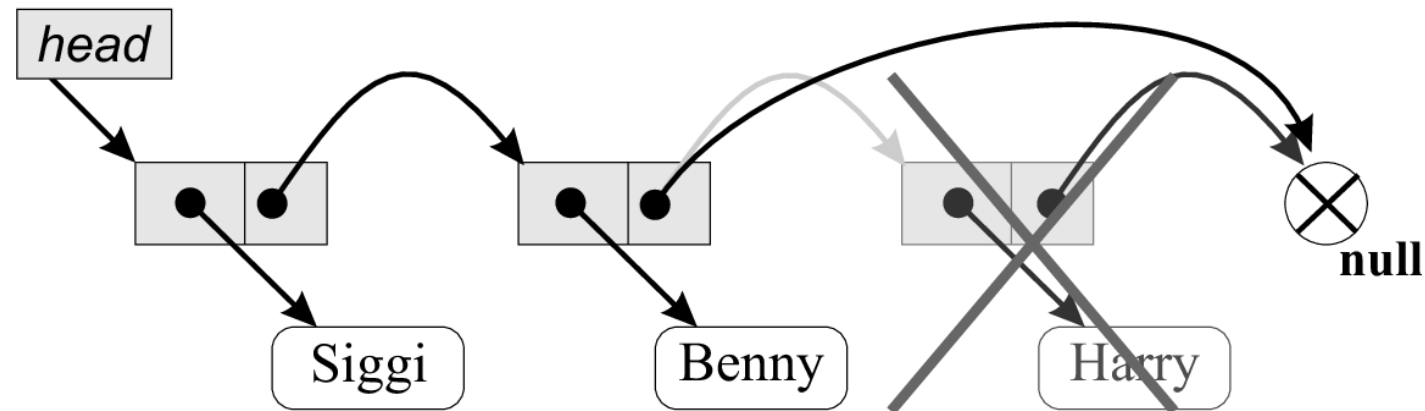


Verkettete Liste: Einfügen am Ende



Verkettete Liste: Löschen

- Beim Löschen des ersten Elements wird der Zeiger head auf den Knoten gesetzt, auf den der erste Knoten verweist.
- Beim Löschen des letzten Elements wird beim vorletzten Knoten der Verweis auf den Nachfolger auf `null` gesetzt.





Komplexität der Operationen

- Bei den Operationen am Ende der Liste muss immer die ganze Liste durchlaufen werden.

Operation	Komplexität
addFirst	1 Aufruf → 1
getFirst	
removeFirst	
addLast	Voller Durchlauf → n
getLast	
removeLast	



Implementierung der Klasse List

- Hilfsklasse Node zur Repräsentation der Knoten mit Verweisen auf das Element (`element`) und den Nachfolger (`next`).
- Zusätzlich zu den beiden Verweisen sind nur noch Methoden zur Manipulation definiert.



Klasse Node

```
public class Node {  
    private Object obj; // Element, Daten  
    private Node next; // Verweis auf den Nachfolger  
  
    public Node(Object o, Node n) {  
        this.obj = o;  
        this.next = n;  
    }  
  
    public Node() {this(null, null); }  
  
    public Object getElement() { return obj; }  
  
    public void setElement(Object obj) { this.obj = obj; }  
  
    public Node getNext() { return next; }  
  
    public void setNext(Node next) { this.next = next; }  
}
```





Klasse List: isEmpty(), addFirst(), getFirst()

```
public class List {  
  
    private Node head = null; // Listenkopf  
  
    public boolean isEmpty() {  
        return head == null;  
    }  
  
    public void addFirst(Object o) {  
        // neuer Knoten als Kopf  
        Node n = new Node(o, head);  
        head = n;  
    }  
  
    public Object getFirst() throws ListEmptyException {  
        if (isEmpty()) {  
            throw new ListEmptyException();  
        }  
        return head.getElement();  
    }  
}  
// ...
```





Klasse List: addLast(), getLast()

```
public void addLast(Object o) {  
    Node n = new Node(o, null);  
    if (head == null) {  
        head = n;  
    } else {  
        Node l = head;  
        // Letztes Element ermitteln  
        while (l.getNext() != null) {  
            l = l.getNext();  
        }  
        l.setNext(n); // neuen Knoten anfügen  
    }  
}  
// ...
```



```
public Object getLast()  
    throws ListEmptyException {  
    if (isEmpty()) {  
        throw new ListEmptyException();  
    }  
    Node l = head;  
    while (l.getNext() != null) {  
        l = l.getNext();  
    }  
    return l.getElement();  
}  
// ...
```





Klasse List: removeFirst(), removeLast()

```
public Object removeFirst()
    throws ListEmptyException {
    if (isEmpty()) {
        throw new ListEmptyException();
    }
    Object o = head.getElement();
    head = head.getNext();
    return o;
}
// ...
```



```
public Object removeLast() throws ListEmptyException {
    if (isEmpty()) {
        throw new ListEmptyException();
    }
    // 1 Element
    if (head.getNext() == null) {
        Object o = head.getElement();
        head = null;
        return o;
    }
    // > 1 Elemente: vorletzten Knoten ermitteln
    Node l = head;
    while (l.getNext().getNext() != null) {
        l = l.getNext();
    }
    Object o = l.getNext().getElement();
    l.setNext(null);
    return o;
}
// ...
```



Klasse List: size()

```
public int size() {  
    int s = 1;  
    if (head == null) {  
        return 0;  
    }  
    // Knoten zählen  
    Node n = head;  
    while (n.getNext() != null) {  
        s++;  
        n = n.getNext();  
    }  
    return s;  
} // ...
```



- Besser: beim Einfügen und Löschen mitzählen – ist effizienter.



Behandlung der leeren Liste

- In der Klasse `List` wird der Listenkopf `head` als Attribut vom Typ `Node` repräsentiert.
- Ausgehend von dieser Referenz sind alle Knoten erreichbar.
- Für den Fall der leeren Liste bedeutet das `head==null`.
- Dies muss bei den Operationen `addLast()` und `removeLast()` stets vorher geprüft werden.
- Alternative (im Buch gezeigt)
 - Durch die Nutzung eines spezialisierten head-Knotens, der als Element `null` enthält und auf das eigentliche erste Element verweist, kann dies vermieden werden.
 - Der head-Knoten wird beim Erzeugen der Liste im Rahmen des Konstruktors angelegt



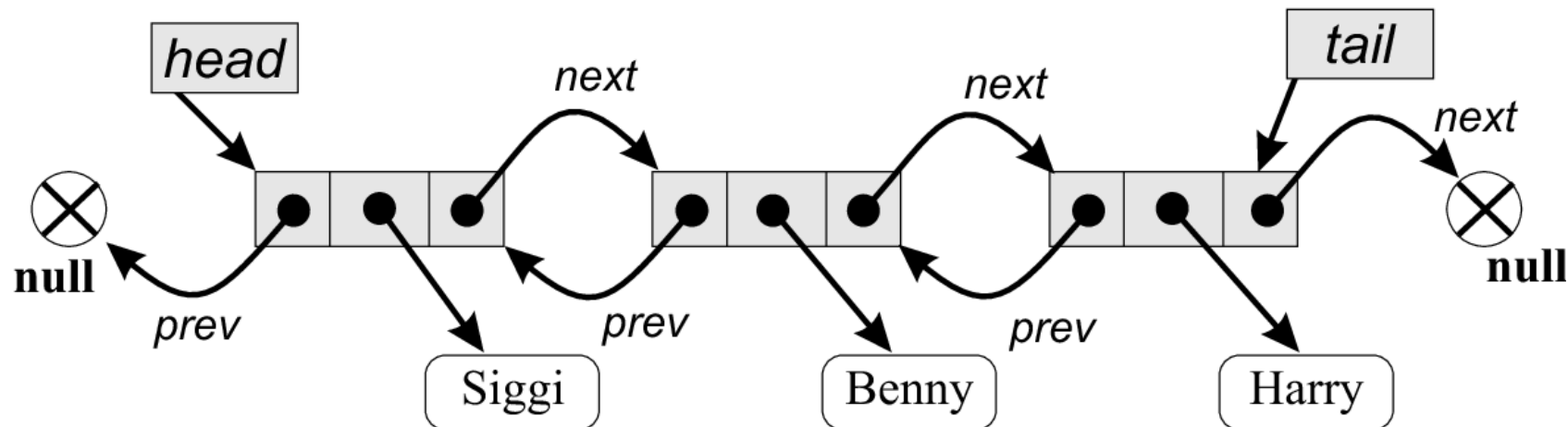
Zugriff auf das letzte Element

- Problem der Implementierung der verketteten Liste ist der Zugriff auf das **letzte Element**.
- Da jeder Knoten nur mit seinem Nachfolger verbunden ist, muss der letzte Knoten erst mit Durchlauf durch die gesamte Liste ermittelt werden. Aufwand: n - Schritte
- Zwar könnte man die Liste um einen Verweis `tail` auf das letzte Element ergänzen.
- Aber beim Löschen des letzten Knotens müsste man immer noch die Liste durchlaufen um den vorletzten Knoten zu suchen

```
public Object getLast()
    throws ListEmptyException {
    if (isEmpty()) {
        throw new ListEmptyException();
    }
    Node l = head;
    while (l.getNext() != null) {
        l = l.getNext();
    }
    return l.getElement();
}
```

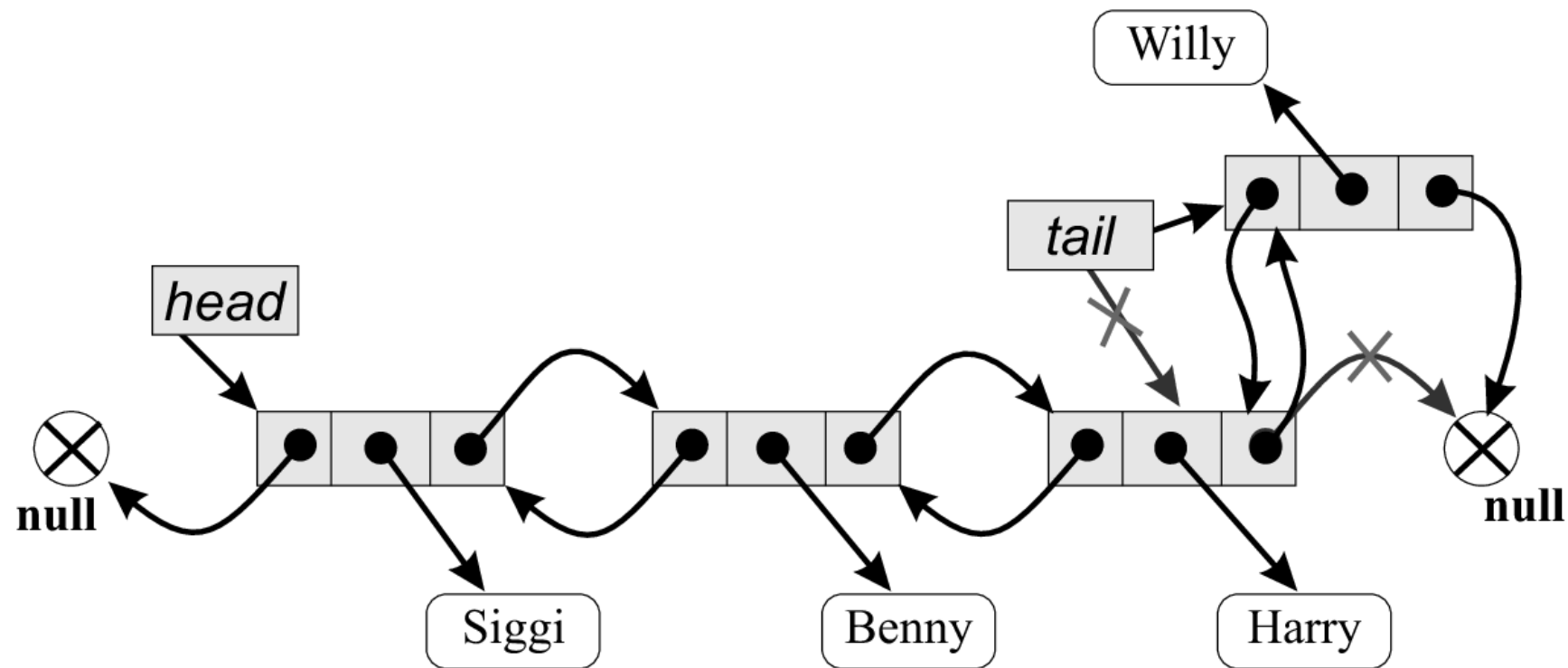
Doppelt verkettete Liste

- einfache Verkettung (nur „Vorwärtszeiger“) erlaubt nur Vorwärtsnavigation
- Rückwärtsnavigation durch zusätzliche „Rückwärtszeiger“: **doppelt verkettete**
- Spezielle Knoten `head`, `tail` für Listenanfang und -ende.





Anhängen am Ende einer doppelt verketteten Liste





Komplexität der Operationen

- Bei den Operationen am Ende der Liste geht es genauso schnell wie am Anfang.

Operation	Komplexität
addFirst	1 Aufruf → 1
getFirst	
removeFirst	
addLast	
getLast	
removeLast	



DList: Klasse Node

```
public class Node {  
    Object obj; // Element, Daten  
    Node prev, next; // Verweis auf Vorgänger, Nachfolger  
  
    public Node(Object o, Node p, Node n) {  
        this.obj = o;  
        this.prev = p;  
        this.next = n;  
    }  
  
    public Node() {this(null, null, null); }  
  
    public Object getElement() { return obj; }  
  
    public void setElement(Object obj) { this.obj = obj; }  
  
    public Node getNext() { return next; }  
  
    public void setNext(Node next) { this.next = next; }  
  
    public Node getPrev() { return prev; }  
  
    public void setPrev(Node prev) { this.prev = prev; } }
```





DList: addFirst(), getFirst()

```
public class DList {  
  
    private Node head = null; // Anfang  
    private Node tail = null; // Ende  
  
    public DList() {  
        head = new Node();  
        tail = new Node();  
        head.next = tail; // Anfang und Ende verknüpfen  
        tail.prev = head;  
    }  
  
    public void addFirst(Object o) {  
        Node n = new Node(o, head, head.next);  
        head.next.prev = n;  
        head.next = n;  
    }  
  
    public void addLast(Object o) {  
        Node l = tail.prev;  
        Node n = new Node(o, l, tail);  
        l.next = n;  
        tail.prev = n;  
    }  
}
```

// ...





DList: getFirst(), getLast(), removeFirst()

```
public Object getFirst() throws ListEmptyException {  
    if (isEmpty())  
        throw new ListEmptyException();  
    return head.next.obj;  
}  
  
public Object getLast() throws ListEmptyException {  
    if (isEmpty())  
        throw new ListEmptyException();  
    return tail.prev.obj;  
}  
  
public Object removeFirst() throws ListEmptyException {  
    if (isEmpty())  
        throw new ListEmptyException();  
    Object o = head.next.obj;  
    head.next = head.next.next;  
    head.next.prev = head;  
    return o;  
}  
// ...
```





DList: getFirst(), getLast(), removeFirst()

```
public Object removeLast() throws ListEmptyException {  
    if (isEmpty())  
        throw new ListEmptyException();  
    Node n = tail.prev;  
    n.prev.next = tail;  
    tail.prev = n.prev;  
    return n.obj;  
}  
  
public int size() { // Knoten zählen  
    int s = -1;  
    Node n = head;  
    while (n.next != null) {  
        s++;  
        n = n.next;  
    }  
    return s;  
}  
  
public boolean isEmpty() { return head.next == tail; }  
// ...
```





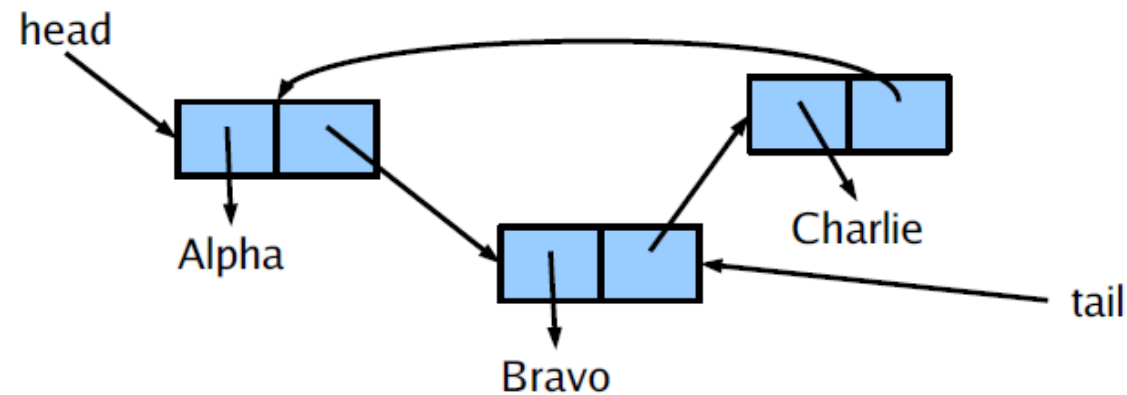
Listen

- Bei einer einfach verketteten Liste mit Tail-Zeiger und Zähler gilt:
 - 1) Alle Operationen sind in einem Schritt erreichbar
 - 2) Es gibt Operationen, die durch die ganze Liste laufen müssen.
 - 3) Löschen am Ende ist nur mit vollem Durchlauf möglich, Einfügen am Ende geht in einem Schritt

- A. 1) und 2)
- B. 2)
- C. 2) und 3)
- D. 3)
- E. Keins davon



Liste



- A. Dies ist eine doppelt verkettete Liste
- B. Die Liste hat 2 Fehler
- C. Das ist kein Problem
- D. Ich verstehe das nicht
- E. Keins davon



a) Zeichnen Sie die einfach verkettete Liste `l` nach folgenden Operationen:

- `l.addFirst("eins"); l.addFirst("zwei"); l.addLast("drei");`

b) Zeichnen Sie die Liste `l` nach folgenden weiteren Operationen:

- `l.removeLast(); o1 = l.getLast(); l.addFirst("vier"); l.addLast("fünf"); o2 = l.getLast();`

c) Welchen Wert hat `o1`? _____

d) Welchen Wert hat `o2`? _____



Komplexität

- Welchen Aufwand erfordert das Einfügen in eine sortierte verkettete Liste?

A. 1

B. $\log n$

C. n

D. $n \log n$

E. n^2